

Checkpoint 4

- **¿Cuál es la diferencia entre una lista y una tupla en Python?**

Tanto una lista como una tupla sirven para almacenar una serie o colección de datos. Sin embargo, cuentan con unas características diferentes. Para comenzar, la sintaxis es distinta en ambos, una lista se delimita usando [], mientras que una tupla se delimita mediante (). Aún así, la principal diferencia es la mutabilidad, esto es, en el caso de las listas, se pueden modificar los datos, mientras que en una tupla no. Para poner un ejemplo:

```
una_lista = [1,2,3]
una_lista[0] = 4
```

Esto modificará la lista y nos lo devolverá como [4,2,3].

```
una_tupla = (1,2,3)
una_tupla[0] = 4
```

Esto nos devolverá un error ya que las tuplas son inmutables. Aun así, hay que tener en cuenta que pese a ser inmutables, una tupla puede contener dentro de sí mismo elementos mutables. Esta característica no es una limitación arbitraria, sino que responde a un diseño orientado a la seguridad y eficiencia.

Esto tiene ciertas ventajas, las tuplas son más eficientes en memoria y ligeramente más rápidas, ya que Python no necesita gestionar posibles cambios en su contenido. Por otro lado, garantizan que los datos permanezcan constantes, lo cual es especialmente útil cuando se trabaja con información que no debe alterarse, como coordenadas, configuraciones o registros fijos.

Otro aspecto relevante es que, aunque una tupla en sí es inmutable, puede contener elementos mutables en su interior, como listas. En ese caso, el contenido interno sí podría modificarse, aunque la estructura de la tupla permanezca intacta.

- **¿Cuál es el orden de las operaciones?**

En Python, para la prioridad de las operaciones se respeta el principio PEMDAS. Esto es, primero se respetan los paréntesis, después van los exponentes, después se realizan las multiplicaciones o divisiones y finalmente las sumas y restas. Por ejemplo, si yo realizo la siguiente operación: $2 + 3 * 4$, la respuesta será 14, ya que se prioriza el $3 * 4$ primero.

Sin embargo, en el mismo caso, si ponemos unas paréntesis, $(2+3) * 4$, la cosa cambia, ya que el orden de las operaciones prioriza las paréntesis primero, haciendo que la suma dentro de la paréntesis se realice primero, seguido de la multiplicación, siendo la en este caso 20. Un aspecto más avanzado es que, cuando varios operadores tienen la misma prioridad (por ejemplo, multiplicación y división), Python evalúa las operaciones de izquierda

a derecha, si tenemos: $20 / 5 * 2$, primero se hace la división y después la multiplicación, siendo la respuesta 8. Hay una excepción en el caso de los exponentes, ya que en caso de tener varios seguidos, $2 ** 3 ** 2$, el orden en este caso va de derecha a izquierda.

- **¿Qué es un diccionario Python?**

Un diccionario Python también es una estructura para almacenar datos, solo que esta vez vienen en pares de “key” o clave y valor. Funciona de la misma forma que en un diccionario de una lengua. En ese caso, buscaríamos una palabra, la cual sería la key o clave, y esta tendría asociada una definición o información, lo cual sería el valor. En un diccionario Python funciona igual, siguiendo la siguiente estructura:

```
un_diccionario = {  
    'edad' : 25,  
    'nombre' : 'Jordan',  
    'trabajo' : 'profesor',  
}
```

Si pedimos buscar el valor de una clave, nos lo dará

`print (un_diccionario['edad'])`, nos devolverá 25. Los diccionarios Python son mutables y se pueden modificar, por ejemplo:

`un_diccionario['edad'] = 30`, esto modificara el valor de la clave 'edad'.
`un_diccionario['ciudad'] = 'Bilbao'`, esto añadira un nuevo par de clave-valor.
`del un_diccionario['trabajo']`, esto eliminará el elemento de 'trabajo' : 'profesor'.

Además, las claves deben ser unicas y no pueden repetirse, si se repiten, el valor anterior se sobrescribe. Las claves deben ser inmutables, siendo strings, números o tuplas.

Sirven para organizar mejor los datos, representándolos de una forma más clara y estructurada y ayudándonos a acceder a ellos más rápidamente.

- **¿Cuál es la diferencia entre el método ordenado y la función de ordenación?**

El método ordenado, `.sort()`, ordena los datos de la lista directamente, modificandola, mientras que la función de ordenación, `sorted()`, devuelve una nueva lista ordenada sin modificar la original. Por ejemplo:

Usando método ordenado:

```
una_lista = [3,1,2]  
una_lista.sort()  
  
print(una_lista)
```

El resultado sería: [1,2,3]

Usando la función de ordenación:

```
una_lista = [3,1,2]
nueva_lista = sorted(una_lista)

print(una_lista)
print(nueva_lista)
```

Los resultados serían: [3,1,2] y [1,2,3].

- **¿Qué es un operador de reasignación?**

Es un tipo de operador que permite modificar el valor de una variable utilizando su propio valor anterior, realizando al mismo tiempo la operación y la asignación en una sola instrucción. Es decir, en lugar de escribir una operación completa donde se repite la variable, estos operadores simplifican el proceso y hacen el código más eficiente y legible.

Su funcionamiento consiste en tomar el valor actual de la variable, aplicar una operación sobre él y guardar el resultado nuevamente en la misma variable. Esto evita la repetición innecesaria del identificador de la variable y reduce la posibilidad de errores.

Los operadores de reasignación que podemos encontrar son:

- += → suma y asigna
- -= → resta y asigna
- *= → multiplica y asigna
- /= → divide y asigna
- //= → división entera y asigna
- %= → módulo y asigna
- **= → potencia y asigna

Por ejemplo:

```
edad = 25

edad +=5

print(edad)
```

El resultado sería 30.

Gracias a estos operadores se puede hacer el código más simple, se mejora la legibilidad y evita repetir la variable. Proporcionan al código una mayor claridad, permitiendo escribir operaciones de forma más limpia y directa, especialmente cuando se realizan cálculos repetitivos sobre una misma variable, haciendo que sea más fácil de leer y entender tanto para el programador original como para otros desarrolladores. Además también contribuyen a la reducción de errores, ya que evitan la repetición innecesaria del

nombre de la variable. Al no tener que escribirla varias veces dentro de una misma expresión, se disminuye la probabilidad de cometer errores de escritura o de referencia, lo que mejora la fiabilidad del código.

Por otra parte, también podemos utilizar operadores de reasignación para valores no numéricos, por ejemplo, en el caso de las cadenas de texto, permiten la concatenación de forma directa:

```
texto = "Hola"  
texto += " mundo"  
print(texto)
```

La respuesta sería: Hola mundo. También se puede usar con listas:

```
lista = [1, 2]  
lista += [3, 4]  
print(lista).
```

La respuesta sería: [1, 2, 3, 4]. La lista se amplía manteniendo su identidad original, lo que significa que no se crea un nuevo objeto, sino que se modifica el existente. Sin embargo, es importante destacar una diferencia interna relevante: no todos los operadores de reasignación se comportan igual a nivel de memoria. Por ejemplo:

```
lista = [1, 2]  
lista = lista + [3]
```

En este caso, se crea una nueva lista, ya que la operación genera un nuevo objeto en memoria. En cambio:

```
lista = [1, 2]  
lista += [3]
```

Aquí, la lista original se modifica directamente en el mismo objeto, lo que puede resultar más eficiente en determinados contextos.